SETUP

The Python and Pandas Development Environment

Jupyter Notebooks

Where data analysts do their work

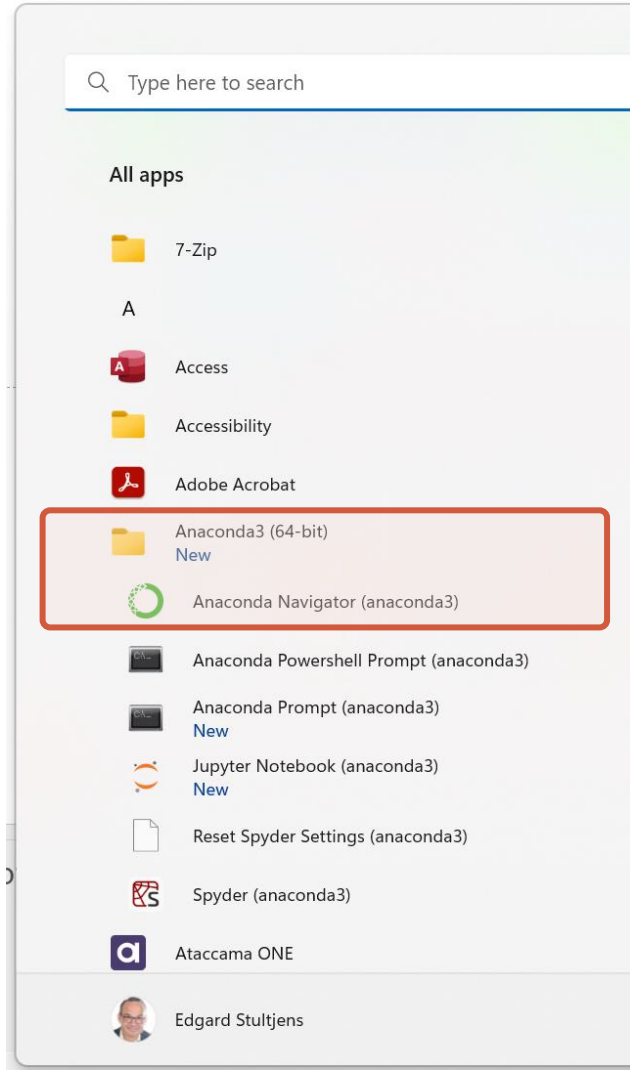
Python Scripts vs Jupyter Notebooks

A comparison

	Python Script	Jupyter Notebook
File Format:	Plain text file	JSON-based notebook file
File Extension:	.py	.ipynb
Code Editor / IDE:	▪ Local: VS Code – PyCharm - Spyder (Anaconda) ▪ Online; Vscode.dev – Replit – GitHub CodeSpaces	▪ Local: Jupyter Notebook – JupyterLab (Anaconda) ▪ Online: Google Colab – Kaggle Notebooks – Jupyter Lite
Contents:	Code + comments	Code + comments + output + links + images
Structure:	Plain text	Cells
Execution Logic:	Linear, top-down	Non-linear, arbitrary order
Executed using:	Command line | IDE | Double-click	Local Jupyter server and browser-based notebook editor
Used for:	fixed / stable routines or pipelines	Data exploration and testing

ANACONDA

- Python and R programming language distribution
- Open source
- Focused on scientific computing
  (data science, machine learning applications, large-scale data processing, predictive analytics, etc.)
- Simplifies package management and deployment
- Virtual environment management
- Supports Windows, Linux and macOS
- Command line (CLI) and GUI support
- 250 Packages automatically installed,
  over 7.500 additional open-source packages can be installed from PyPI
- ⊞ ► Anaconda ► Anaconda Navigator



Modules, Packages and Libraries

- Facilitate and promote code reuse ► increased productivity
- Module: related code saved together in one file
- Package: collection of modules
- Library: group of related modules and packages
- “Package” and “library” are often used interchangeably

Popular Python Tools and Libraries

- Data Analyis
  - ➔ Pandas, NumPy, Scipy
  - ➔ Read and write data, numerical integration, interpolation, linear algebra
- Machine Learning
  - ➔ TensorFlow, Pytorch, Keras, Scikit Learn
  - ➔ Classification, regression, clustering, predictive analytics, building and training deep learning models
- Data Visualization
  - ➔ Visualizations, dashboards, widgets
  - ➔ Matplotlib, plotly, bokeh, Streamlit

CONDA
& CONDA Workflow

Getting Started

Conda

- Central **package repository** manager
- **Dependency manager**
  Pandas ► NumPy, Cython, blas, certify, six, etc.
- **Environment manager**: create reproduceable code environment using a YAML configuration file, i.e. environment.yml ► all required packages in correct version
- Open source
- Supports all programming languages, e.g. Python, R, Java, C, C++
- Similar to PIP, the Python “Preferred Installer Program”

Best practice: use separate environment for every project, never use Base!

Conda Workflow

5 Step process

1. Create environment
2. Activate environment
3. Install packages
4. Launch JupyterLab
5. Deactivate environment

2 Interfaces:

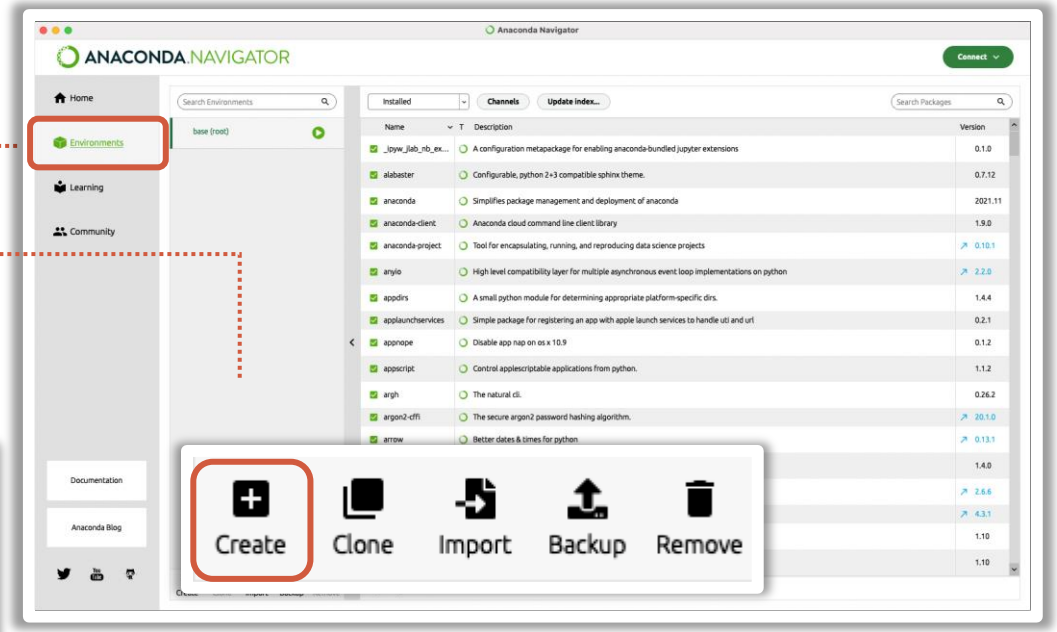
- **CLI** or Command Line Interface aka *“the prompt”*
- **GUI** or Graphical User Interface aka “Anaconda Navigator”

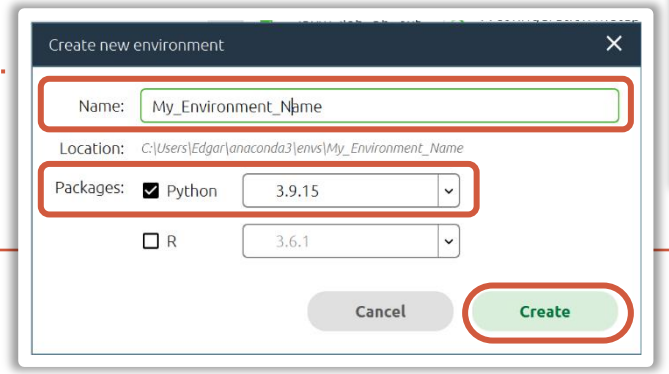
Anaconda Navigator

Managing Environments “The Easy Way”

Anaconda Navigator: Create Environment

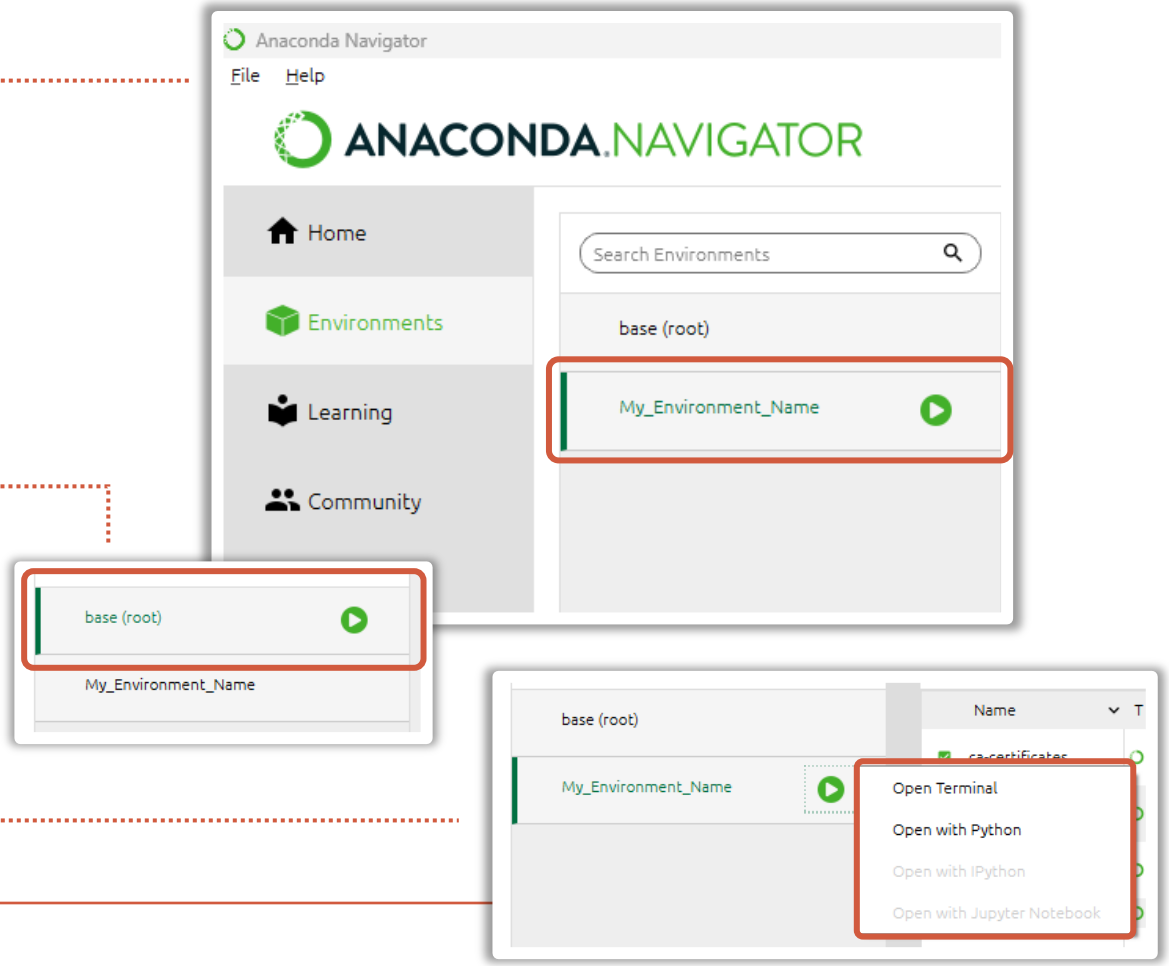
- Open Anaconda Navigator
  - Mac/Linux: Open Launchpad ► click Anaconda-Navigator icon
  - Windows: open Anaconda Command ► Anaconda ► Anaconda Navigator
- Activate the Environments tab
- Click the [Create] button
- Enter name (no spaces or special characters) and optionally select Python Version
- Click [Create]





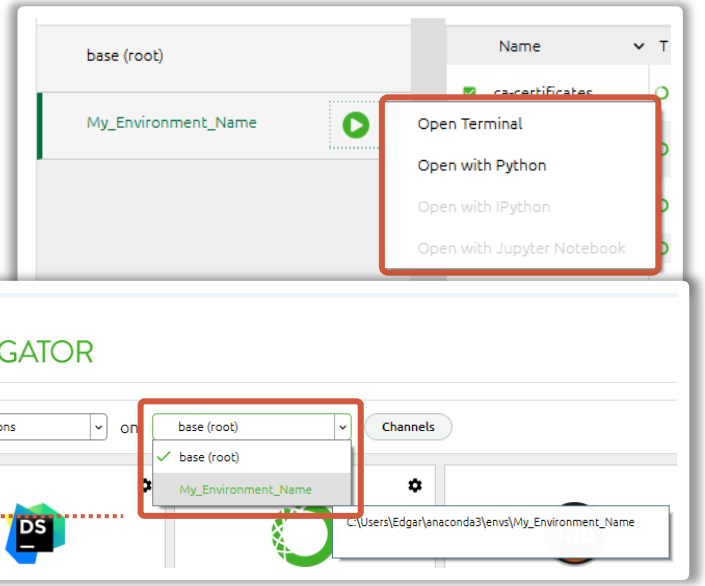
Anaconda Navigator: Activate Environment

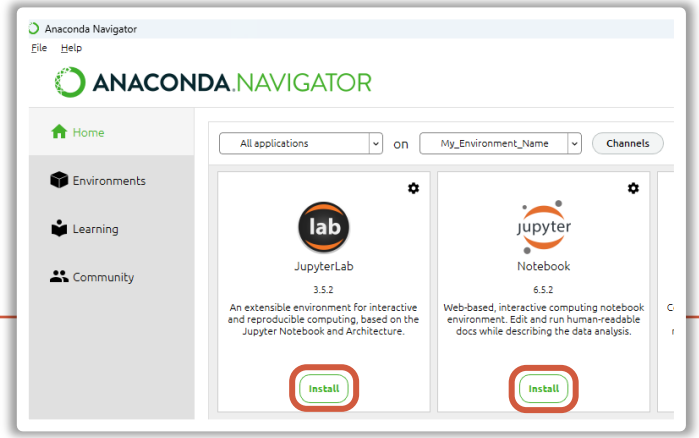
- Check active environment
- Click on the environment name to be activated, e.g. Base (root)
- The new active environment will now show the green play button
- Click the play button
- Select the tool to open the environment in



Anaconda Navigator: Install Jupyter Notebook

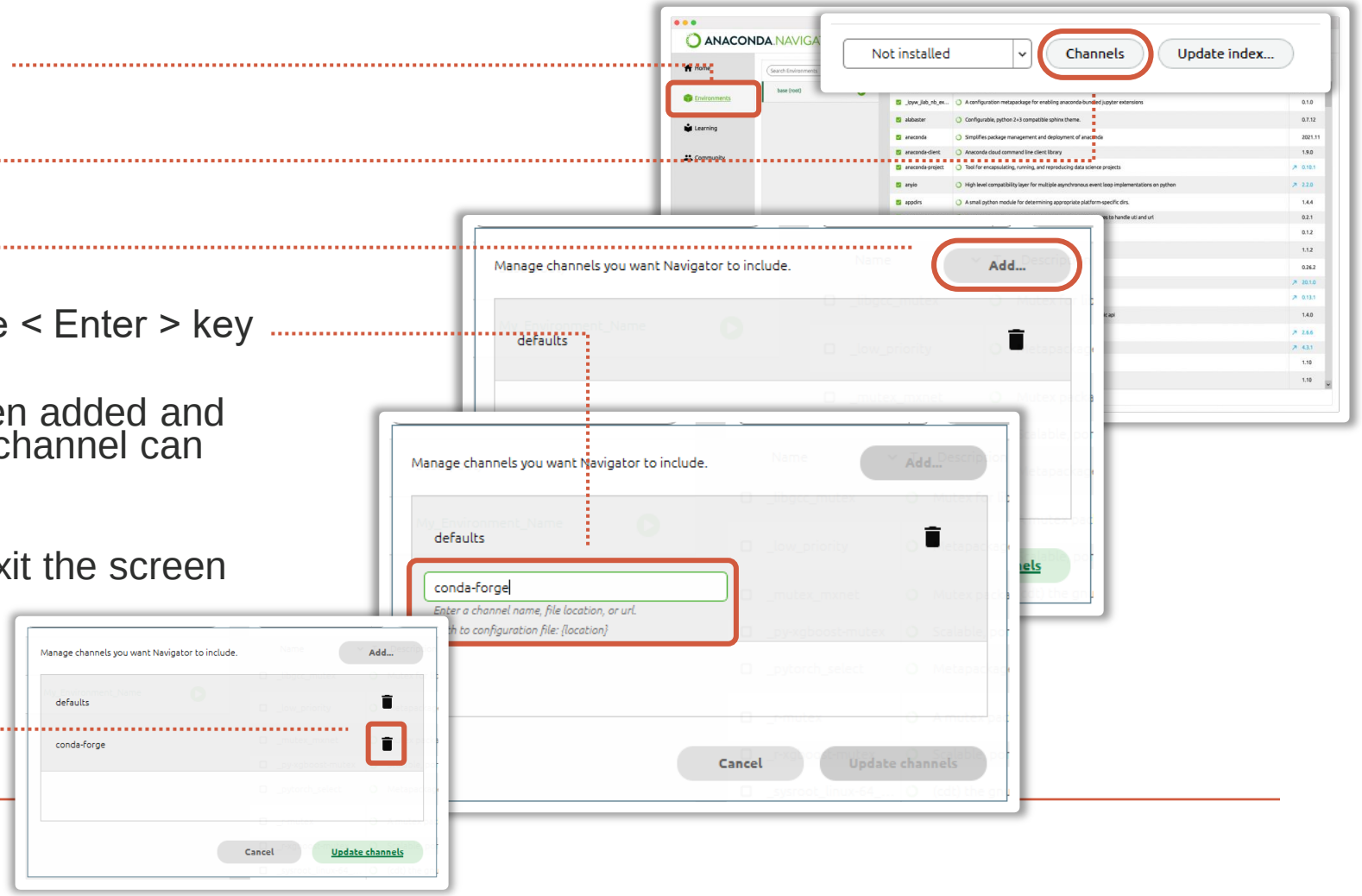
- By default Jupyter Notebook and JupyterLab are NOT available for new environments
- To make this option available, go to the Home tab
- Make sure your target environment is active
  Select it from the dropdown if not
- Now Install either JupyterLab or Jupyter Notebook by clicking the appropriate [Install] button





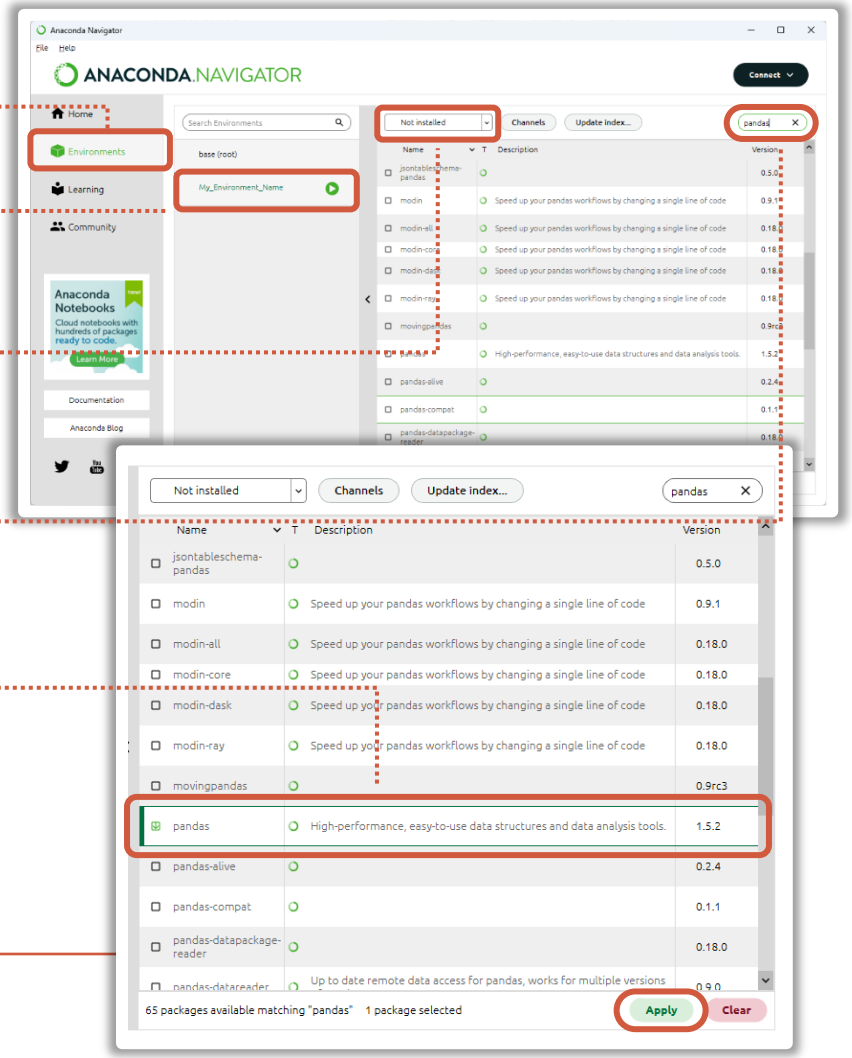
Anaconda Navigator: Adding Channels

- Activate the Environments tab
- Click the [Channels] button
- Click the [Add] button
- Enter “conda-forge” and hit the < Enter > key
- The new channel has now been added and packages hosted on this new channel can now be installed
- Click [Update Channels] to exit the screen
- To remove a channel simply click on the trashcan icon next to it



Anaconda Navigator: Adding Packages

- Activate the Environments tab
- Select the environment you want to add the package to
- Change the package filter to “Not installed”
- Enter the name of the package you want to install in the search box and hit < Enter >
- Select the package to install from the search result list and click the [Apply] button to initiate the installation



Exercise ENV-01

Creating a Pandas project

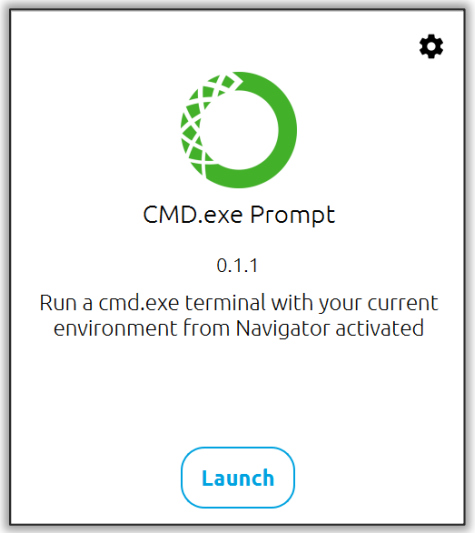
- Open Anaconda Navigator
- Create a new environment called “My first Pandas project”
- Activate the new environment
- Install the Pandas library

CONDA CLI

The Techie Way

Conda CLI: Create Environment

- Open CLI:
  - Mac/Linux: open terminal window
  - Windows: open Anaconda Command
    ⊞ ▶ Anaconda ▶ Anaconda Navigator ▶ CMD.exe Prompt



Check Conda is installed and active
`conda --version` ▶ `conda 4.12.0`

```
(base) ➜  ~ conda --version
conda 4.12.0
```

Check active environment
`conda env list`

```
(base) ➜  ~ conda env list
# conda environments:
#
base                  *  /Users/ienglander/opt/anaconda3
```

Update Conda to the latest version
`conda update -n base conda`

See also: Conda Cheat Sheets online | Moodle

Conda CLI: Create Environment

- Create new environment for your project
  `conda create --name project_name`
  ► Confirm by typing ‘y’ when prompted

```
(base) ➜  ~ conda create --name example
Collecting package metadata (current_repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: /Users/ienglander/opt/anaconda3/envs/example


Proceed ([y]/n)? y

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#     $ conda activate example
#
# To deactivate an active environment, use
#
#     $ conda deactivate

(base) ➜  ~
```

- Create new environment using specific python version
  `conda create --name project_name python=3.9`

Conda CLI: Activate Environment

- Check your environment is available
  `conda env list`

```
(base) ➜  ~ conda env list
# conda environments:
#
base                  *  /Users/ienglander/opt/anaconda3
example                  /Users/ienglander/opt/anaconda3/envs/example
```

- Activate target environment
  `conda activate example`

  ► The new environments name is now displayed between brackets at the start of the command line

```
(base) ➜  ~ conda activate example
(example) ➜  ~
```

Conda CLI: Install Packages

- Check installed packages
  `conda list`

```
(example) ➜  ~ conda list
# packages in environment at /Users/ienglander/opt/anaconda3/envs/example:
#
# Name                    Version                   Build  Channel
```

- Search for packages
  `conda search` *`fuzzy_package_name`* `[--info]`

- Install additional packages
  `conda install package_name`
  ► Confirm by typing '`y`' when prompted

```
#
base                  *  /Users/ienglander/opt/anaconda3
example                  /Users/ienglander/opt/anaconda3/envs/example

(base) ➜  ~ conda activate example
(example) ➜  ~ conda list
# packages in environment at /Users/ienglander/opt/anaconda3/envs/example:
#
# Name                    Version                   Build  Channel
(example) ➜  ~ conda install jupyterlab dask pandas hvplot
Collecting package metadata (current_repodata.json): done
Solving environment: done
```

- Install packages not found in standard repository
  `conda install -c channel_name package_name`

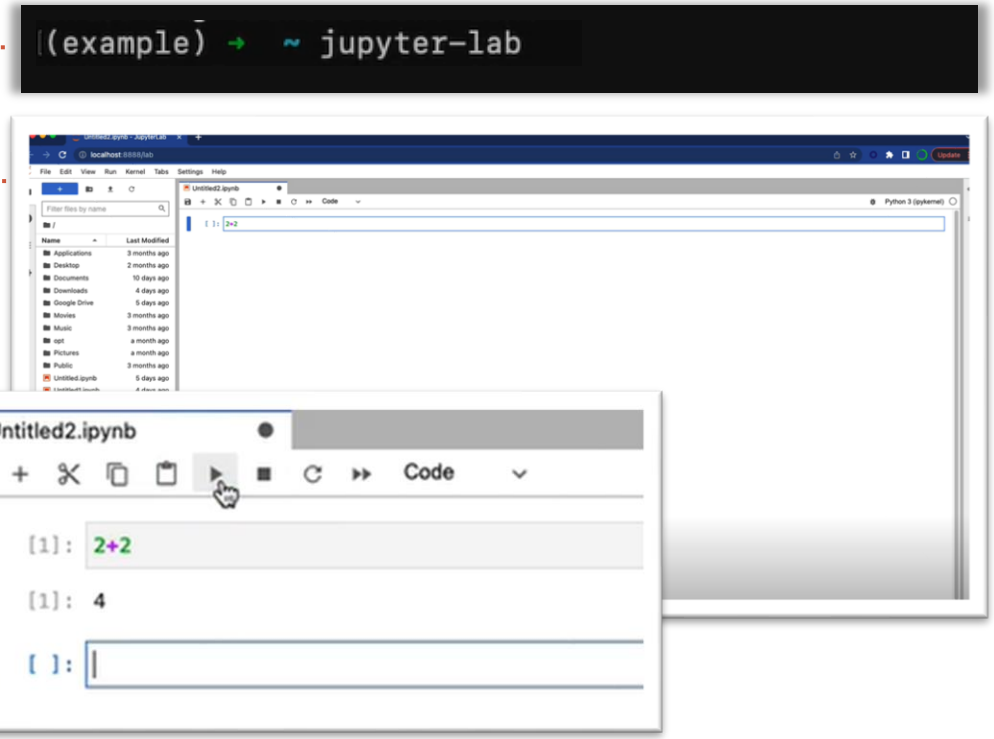
  Example: `conda -c conda-forge condastats`

Conda CLI: Launch JupyterLab

- Launch JupyterLab by running...
  `jupyter-lab`

- JupyterLab opens in browser

- Test by entering simple math equation in first cell and clicking the run button ▶ or hitting SHIFT-ENTER



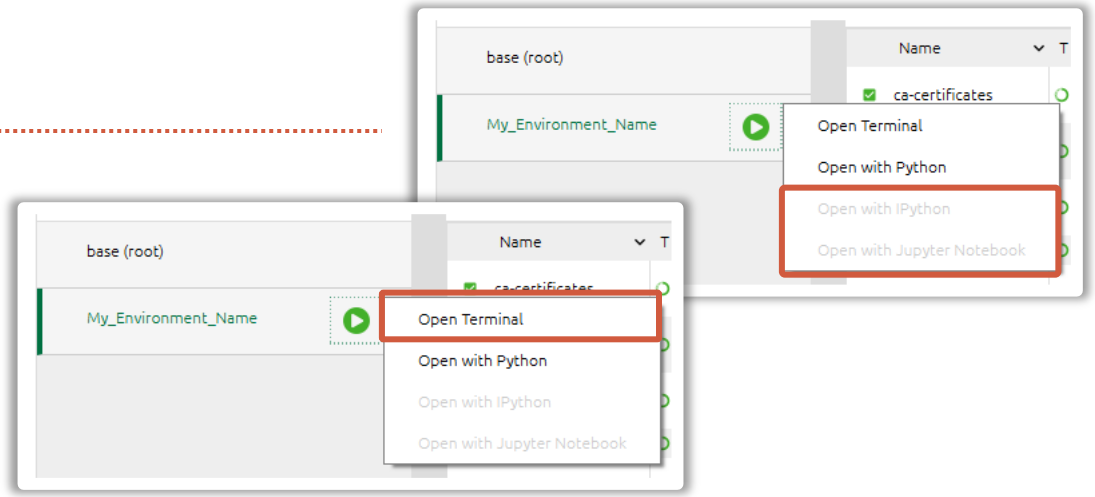
Conda CLI:
Deactivating Sessionns and Environments

- End JupyterLab session
  `CTRL+C`
  - Confirm by typing ‘y’ when prompted

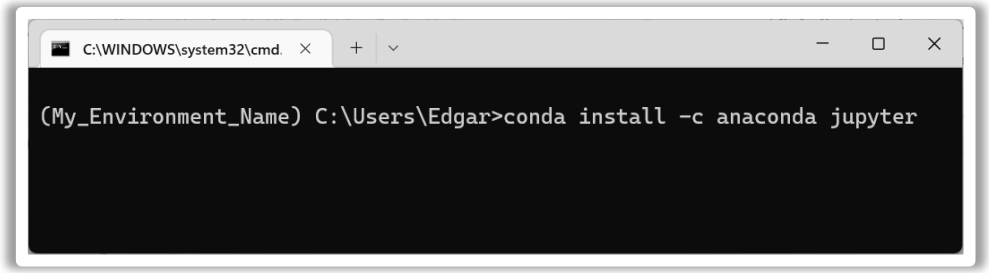
BEST PRACTICE: Always deactivate environment before activating another one
`Conda deactivate`

Conda CLI: Install Jupyter Notebook

- By default Jupyter Notebook and JupyterLab are NOT available for new environments
- To make this option available, open the Terminal for the environment



- Run `conda install -c anaconda jupyter`
  - ► Confirm by typing ‘y’ when prompted



Exercise CLI-01

Switching environments using the CLI

- Open Anaconda Navigator
- Open the command prompt and check…
  - The active environment
  - The version of conda that is installed
  - What the available environments are
- Use the CLI to switch to another environment

Exercise CLI-02

Installing a library using the CLI

- Open the command The active environment
- Switch to the “My first Pandas project” environment if needed
- Search the correct package name for the Pandas Profiling library
- Install Pandas Profiling

GIT

Playing it safe….

Source: https://xkcd.com/1597/

Version Control Systems (VCS)

- **Track changes** in any set of files
- **Coordinate work** among programmers collaboratively developing source code during software development
- Support **code review** process
- **Roll back** to older version in case of issues
- Maintain multiple **versions**
- Assist with bug fixing by identifying impacted versions

Categories of Version Control Systems

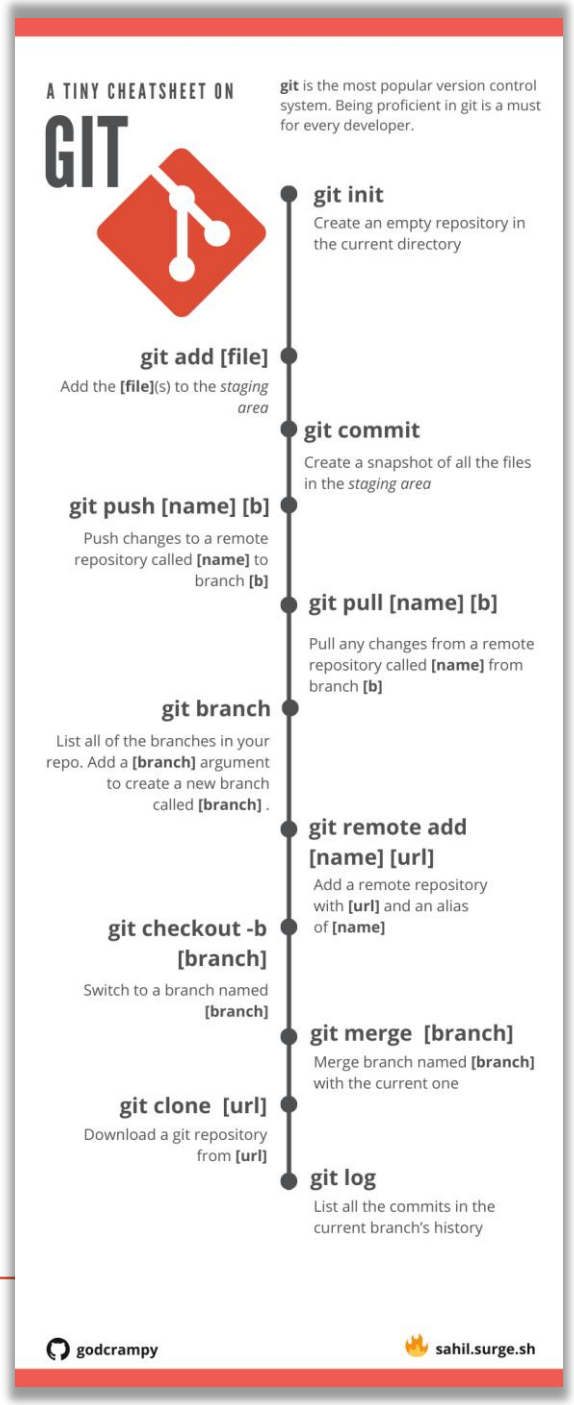
- **Local** Version Control
  - Single local database to track and store changes to files
- **Centralized** Version Control
  - Single server to track and store all changes and file versions for multiple users
  - Less complex, code and work is shared by default, centralized control
  - Single point of failure: nobody works if server goes down
  - Examples: Team Foundation Server, Subversion
- **Distributed version** control system
  - Single centralized repository with mirrored copies on client side
  - Central repository can be rebuilt using local copies
  - Examples: git, Mercurial

Using git

- **Command line**
  - Most flexible: all capabilities supported
  - Fast
  - Not easy for beginners
- **Code editors and IDEs**
  - Basic built-in support
  - Extensions, e.g. GitLens for VSCode
- **GUI clients**
  - See overview on git-scm.com
  - Not as powerful or flexible as command line
  - Most popular: GitHub Desktop, SourceTree, GitKraken (free for open source)

git

- Designed by **Linus Torvald** – Mr Linux – in 2005
- Git means…
  - "unpleasant person" in British English slang
  - "Global information tracker" when it works
  - "Goddamn idiotic truckload of sh*t" when it breaks.
- Distributed version control system (VCS)
- Handles large projects **efficient**ly
- Powerful **branching and merging** capabilities
- **Open Source**



Source: Git Cheat-Sheet [infographic + pdf] - DEV Community

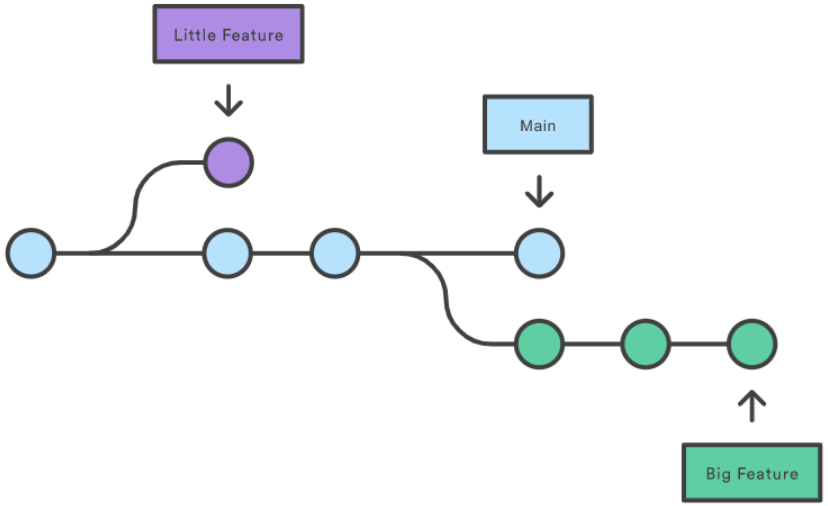
GitHub

- Internet hosting service for software development and version control using git
- Provides Git's distributed version control capabilities – **git Remote Repository** – plus…
  - Access control,
  - Bug tracking,
  - Software feature requests,
  - Task management,
  - Continuous integration, and
  - Wikis for every project
- Commonly used to host **open source software** development projects
- Largest source code host as of November 2021
- Subsidiary of Microsoft since 2018

Git Workflow

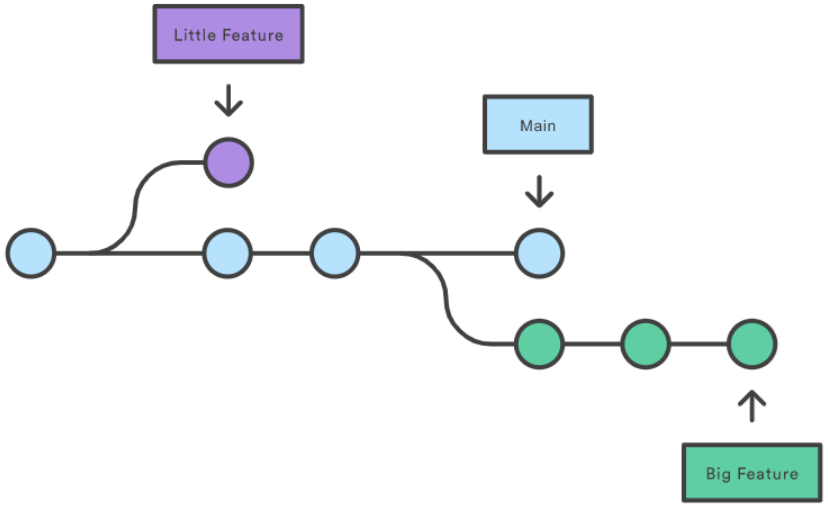
- **Working Directory**: actual code / files
- **Staging Area** aka **Index**:
  - All changes – additions | updates | deletes – proposed for next commit (`git add`)
  - User reviews changes before commit here
- **Local Repository**
  - Permanently stores snapshot of each commit (`git commit`)
  - Include meaningful comment (`git commit –m “Meaningful comment”`) to document change history; use without –m to open default editor and add lengthy comment
  - Full snapshots incl ID, message, datetime and author (no duplicates, compression)
- **Remote Repository**: shared snapshot store
  - Push changes to shared repository (`git push`)

Git Branches



- **Main** aka **Master**: a repository's "default" branch
- Branch: a snapshot / copy of Main
- Changes are applied to branch
- Branch is merged back with Main when done by raising pull requestr

Git Forks



- **Main** aka **Master**: a repository's "default" branch
- Branch: a snapshot / copy of Main
- Changes are applied to branch
- Branch is merged back with Main when done by raising pull requestr

Git Changes

Branches vs Forks

Branch

- A snapshot / copy of Main
- Goal:
- Changes are applied to branch
- Branch is merged back with Main when done by raising pull requester
- User needs to be granted access to the repository
- **Main** aka **Master**: a repository's "default" branch

Fork

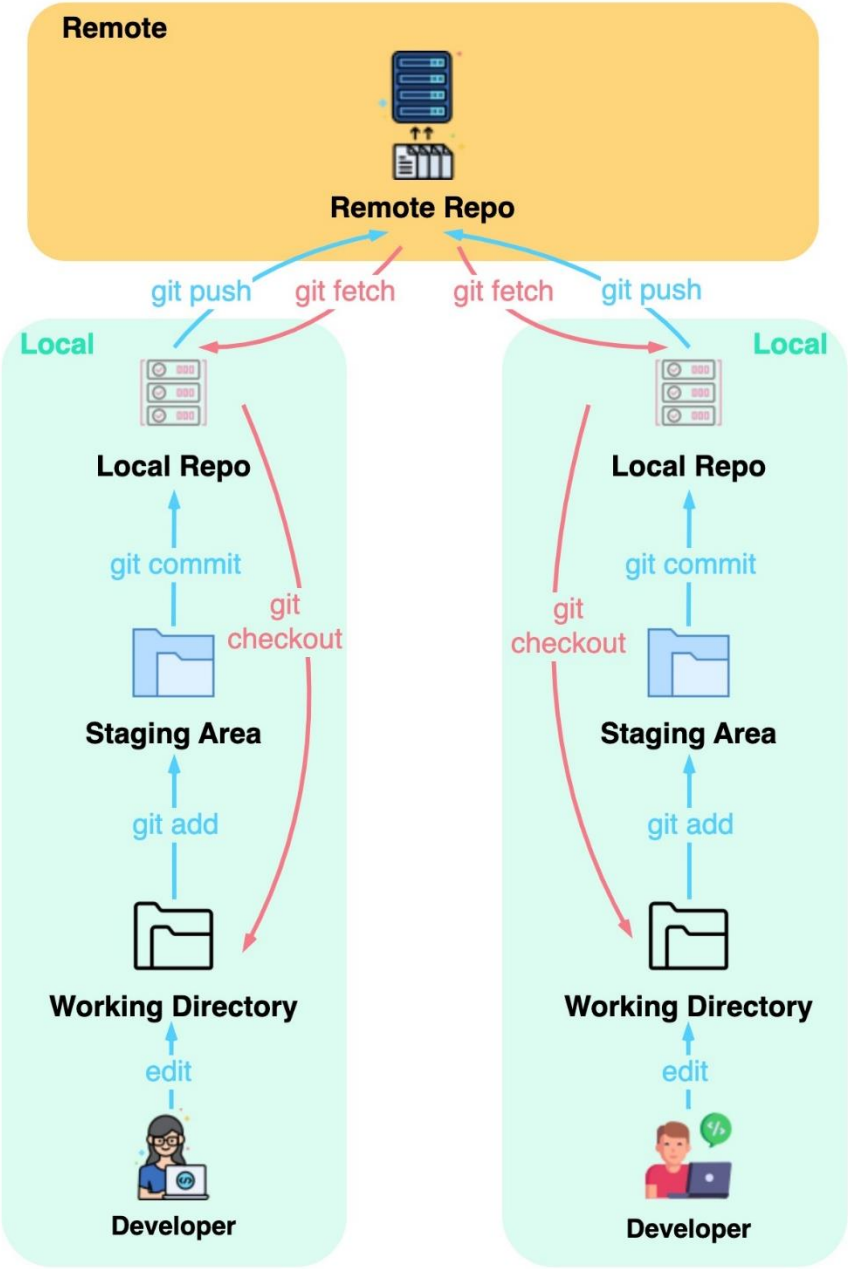
Git Changes

Branches vs Forks

	Branch	Fork
What?	Project **clone**	Independent **copy**
Goal	Allow developers on a project to work on changes without impacting other developers	Create full copy of the repo for separate development
Single remote repository?	☑	☒
Link to original?	☑	☒
Type of contributor	Internal	External
Registered user?	☑	☒
Complexity	Medium	High
Changing the original	Merge through Commit	Pull request

COMMON GIT COMMANDS

- `git init` ▶ turns a directory into an empty Git repository
- `git add` ▶ adds files | file masks | folder to the staging area
- `git commit` ▶ record the changes made to the local repository
- `git status` ▶ returns the current state of the repository
- `git clone` ▶ copy and download a remote repository to local computer
- `git fetch` ▶ imports latest commits in remote branch – no merge
- `git diff` ▶ identify differences between working directory and staging
- `git merge` ▶ merge branch changes into main branch
- `git pull` ▶ git pull + git merge
- `git push <remote_URL/remote_name> <branch> | - all`
  ▶ Sends local commits to the remote repository



Source: EP 40: Git workflow - by Alex Xu - ByteByteGo Newsletter

GIT
Cheat Sheet

Git Cheat Sheet

REBELLABS by ZEROTURNAROUND

Create a Repository

From scratch -- Create a new local repository
```
$ git init [project name]
```

Download from an existing repository
```
$ git clone my_url
```

Observe your Repository

List new or modified files not yet committed
```
$ git status
```

Show the changes to files not yet staged
```
$ git diff
```

Show the changes to staged files
```
$ git diff --cached
```

Show all staged and unstaged file changes
```
$ git diff HEAD
```

Show the changes between two commit ids
```
$ git diff commit1 commit2
```

List the change dates and authors for a file
```
$ git blame [file]
```

Show the file changes for a commit id and/or file
```
$ git show [commit]:[file]
```

Show full change history
```
$ git log
```

Show change history for file/directory including diffs
```
$ git log -p [file/directory]
```

Working with Branches

List all local branches
```
$ git branch
```

List all branches, local and remote
```
$ git branch -av
```

Switch to a branch, my_branch, and update working directory
```
$ git checkout my_branch
```

Create a new branch called new_branch
```
$ git branch new_branch
```

Delete the branch called my_branch
```
$ git branch -d my_branch
```

Merge branch_a into branch_b
```
$ git checkout branch_b
$ git merge branch_a
```

Tag the current commit
```
$ git tag my_tag
```

Make a change

Stages the file, ready for commit
```
$ git add [file]
```

Stage all changed files, ready for commit
```
$ git add .
```

Commit all staged files to versioned history
```
$ git commit -m "commit message"
```

Commit all your tracked files to versioned history
```
$git commit -am "commit message"
```

Unstages file, keeping the file changes
```
$ git reset [file]
```

Revert everything to the last commit
```
$ git reset --hard
```

Synchronize

Get the latest changes from origin (no merge)
```
$ git fetch
```

Fetch the latest changes from origin and merge
```
$ git pull
```

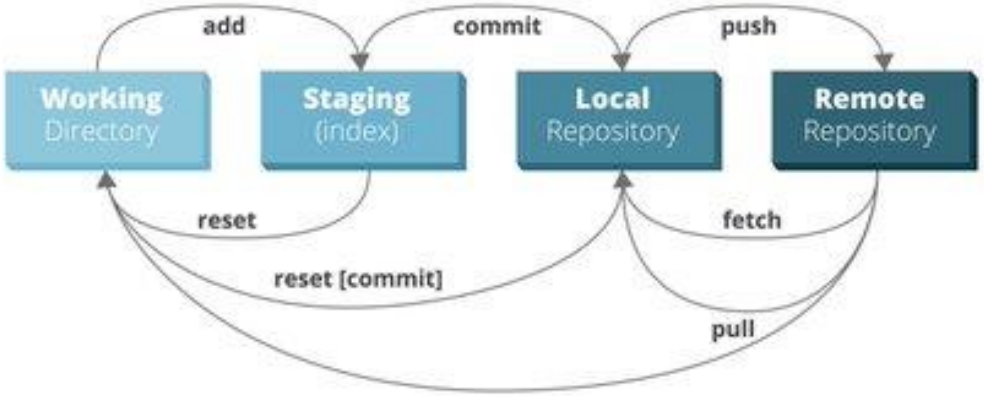
Fetch the latest changes from origin and rebase
```
$ git pull --rebase
```

Push local changes to the origin
```
$ git push
```

Finally!

When in doubt, use git help
```
$ git command --help
```

Or visit https://training.github.com/ for official GitHub training.



Source: https://www.jrebel.com/blog/git-cheat-sheet

Git Best Practices

- Commit often, but: not too small, not too big…
  - 5 – 10 commits / day
- Always commit related changes only
  - 1 Thing at a time… don't mix bugs, enhancements, etc.
  - Think snapshot, think roll-back
- Always use branches
  - New features, bug fixes, ideas, …
- Only commit completed work
  - Test before you commit!
- Make it meaningful… No cryptic commit comments!
  - Past tense, e.g. fixed bug

GitHub Desktop

- Download for either Windows or Mac from GitHub Desktop
  - See Software installation instructions
- Local app for github.com
- Easier on beginners
  - Graphical User Interface (GUI)

Exercise Git-01

Creating a new repository on GitHub

- Goto github.com
  - Either register or log in
- Create a new repository, e.g. “My first remote repository”
  - Choose Public or Private
  - Add a README file
  - Add a gitignore file for python
  - Choose a license type if you chose Public above, e.g. MIT license is the most liberal
- Add a file to the repository and commit the change
- Edit the ReadMe.md inside Github to include your name and commit

Exercise Git-02

Cloning your repository to your computer

- Goto github.com
  - Either register or log in
- Clone “My first repository repository” to your computer
- On your computer, update the "ReadMe.md" file to also include the city you live in
- Commit the change you have made
  - Make sure you add a meaningful comment
- Push your change to Github
- Check your changes on Github.com

Exercise Git-03

Cloning someone else's repository

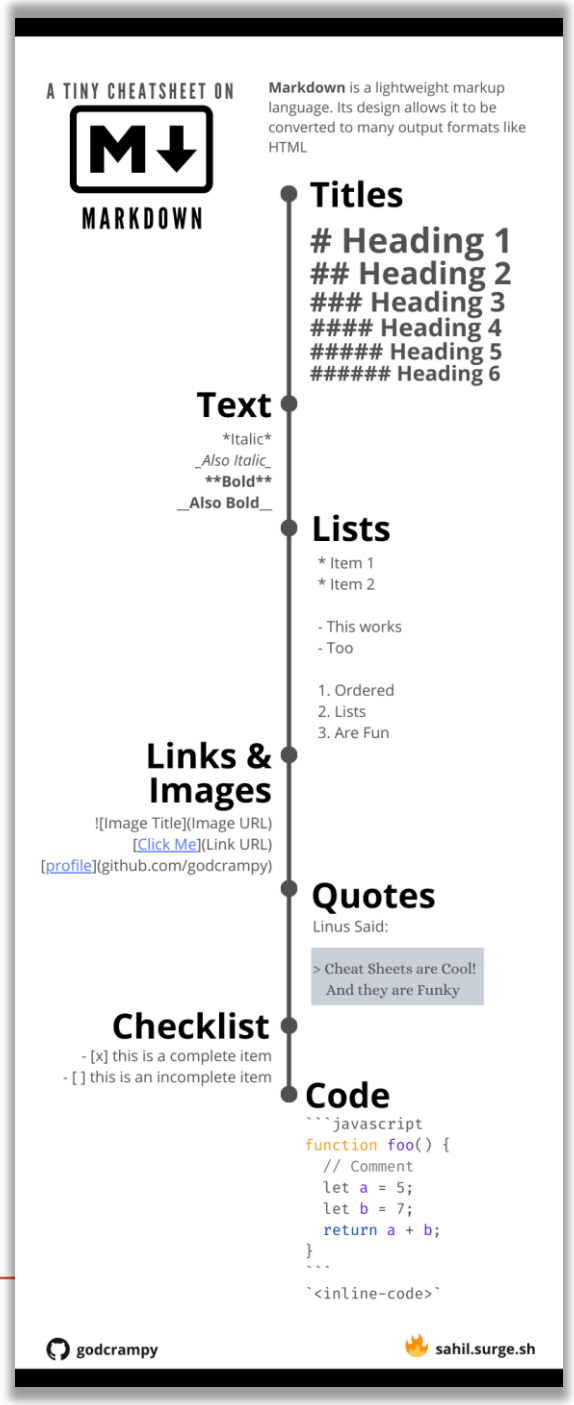
- Open Github Desktop
- Clone https://github.com/EdgardStultjens/My-First-Remote-Repository.git
- Inspect the contents on your local machine
- Add a *YourName*.md file and push it to the remote repository
- Update the README.md and add your name to the student list

Markdown

Formatting plain tekst files

Markdown

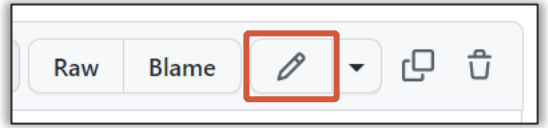
- Lightweight markup language
- Used for creating formatted text using a plain-text editor
- Created by John Gruber and Aaron Swartz 2004
- Widely used in blogging, instant messaging, online forums, collaborative software, documentation pages, and readme files
  - Used by websites like e.g. Bitbucket, GitHub, OpenStreetMap, Reddit, SourceForge, and Stack Exchange
  - Used by tools like e.g. Jupyter Notebook, MS Teams, Discord, RStudio and Obsidian
- Multiple variants, e.g. GitHub Markdown, Markdown Extra, LiaScript
- Used on GitHub for formatting readme files in support of project and user profiles
- See [Cheat Sheet on Moodle]

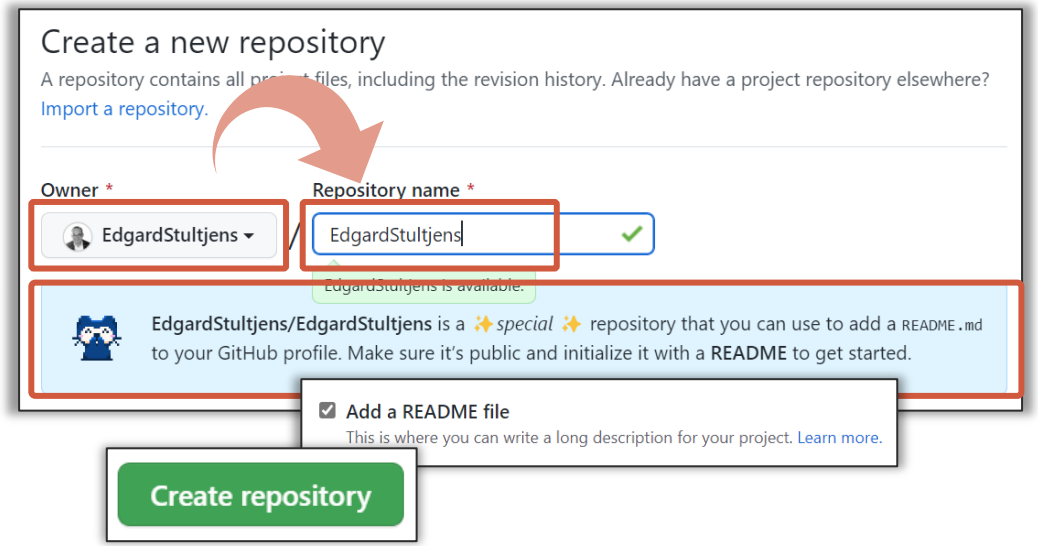


Source: [Markdown Cheat-Sheet [PDF + Infographic]]

Updating your GitHub Profile

- Go to github.com
- Create a new public repository
  - ! Name: your GitHub user name
  - Add a README file
  - Click < Create Repository >
- Click the Penci icon to switch to Edit mode





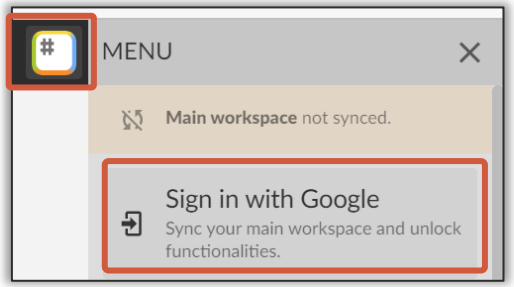
- Tweak it to your taste using e.g. …
  - A regular Mardown editor, e.g. StackEdit.io
  - The GitHub Profile Generator
  - The awesome list of good looking GitHub Profiles

    TIP: to access the profile code,
    - Open the readme by clicking the link
    - Click the [Raw] button
    - Copy paste at will

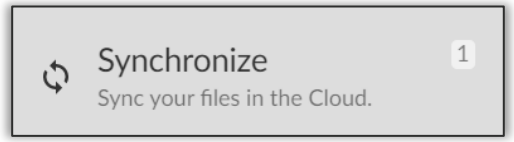
    Note: some of these profiles use plain html instead of Markdown… Keep it simple for now!
- Need some help and inspiration? See How to create an amazing profile with GitHub

Using StackEdit.io

- Go to stackedit.io
- Click the icon top right and select “Sign in with Google”



- Open the main menu again and now select “Synchronize” This will ensure your markdown files get backed up to your Google Drive



- There appears to be an issue connecting to a GitHub account, so for now we will just manually copy over the code from the GitHub README file

Assignment Git-04

Create your GitHub Profile

- Go to github.com
- Create your profile repository
- Tweak it to your taste using e.g. …
  - A regular Mardown editor, e.g. StackEdit.io
  - The GitHub Profile Generator
  - The awesome list of good looking GitHub Profiles

    TIP: to access the profile code,
    - Open the readme by clicking the link
    - Click the [Raw] button
    - Copy paste at will
      Note: some of these profiles use plain html instead of Markdown… Keep it simple for now!
- Need some help and inspiration? See How to create an amazing profile with GitHub

Jupyter Notebook

The data analyst’s (integrated) development environment

JupyterLab Interface

Menu Bar

File Browser

Running terminals and kernels

Table of Contents

Extension Manager

Properties Inspector

Debugger

File Edit View Run Kernel Tabs Settings Help

Filter files by name

/ data /

Name	Last Modified
1024px-Hubble_In...	an hour ago
bar.vl.json	an hour ago
Dockerfile	an hour ago
iris.csv	an hour ago
japan_meterologic...	an hour ago
Museums_in_DC....	an hour ago
README.md	an hour ago
Untitled.ipynb	an hour ago
untitled.txt	35 minutes ago
zika_assembled_g...	an hour ago

Data.ipynb

Open a CSV file using Pandas

```
[4]: import pandas
df = pandas.read_csv('../data/iris.csv')
df.head(20)
```

[4]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	se
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
5	5.4	3.9	1.7	0.4	setosa
6	4.6	3.4	1.4	0.3	setosa
	5.0	3.4	1.5	0.2	setosa
	4.4	2.9	1.4	0.2	setosa
	4.9	3.1	1.5	0.1	setosa
	5.4	3.7	1.5	0.2	setosa
	4.8	3.4	1.6	0.2	setosa
	4.8	3.0	1.4	0.1	setosa
	4.3	3.0	1.1	0.1	setosa
	5.8	4.0	1.2	0.2	setosa
	5.7	4.4	1.5	0.4	setosa
	5.4	3.9	1.3	0.4	setosa

jupyterlab.md

JupyterLab Demo

JupyterLab: The next generation user interface for Project Jupyter

https://github.com/jupyter/jupyterlab

It started as a collaboration between:

- Project Jupyter
- Bloomberg
- (then) Continuum

and now involves many other people from many other places (not purely academic or business)

bar.vl.json | 1024px-Hubble_Interac

Mode: Command | Ln 1, Col 1 | Data.ipynb

SEARCH

- New Console
- ✓ Enable Extension Manager
- ✓ Autosave Documents
- Download
- Open from Path...
- Reload Notebook from Disk
- Revert Notebook to Checkpoint
- Save Notebook ⌘ S
- Save Notebook As... ⇧ ⌘ S
- Show Active File in File Browser
- Jupyter Reference

[ ]:

Cell Operations

Duplicate

Move Up/Down | Insert Above/Below

Delete

Command Palette

Command/Ctrl Shift C

JupyterLab Keyboard Shortcuts

• edureka! •

JUPYTER NOTEBOOK CHEAT SHEET

Learn PYTHON from experts at https://www.edureka.co

Jupyter Notebook

Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations and narrative text. It is used for data cleaning and transformation, numerical simulation, statistical modeling, data visualization, machine learning, and much more.

Saving/Loading Notebook

File | Edit | View

- New Notebook → Create new Notebook
- Open... ← Open an existing Notebook
- Make a Copy... → Make copy of the current Notebook
- Save as... ← Save Current Notebook
- Rename... → Rename current Notebook
- Save and Checkpoint ← Save Current Notebook & record Checkpoint
- Revert to Checkpoint → Revert Notebook to a previous Checkpoint
- Print Preview ← Preview of the printed Notebook
- Download as → Download Notebook as IPython Notebook, Python, HTML, Markdown, PDF
- Trusted Notebook
- Close and Halt ← Close Notebook & stop running scripts

Edit Cells

Edit | View | Insert

- Cut Cells → Cut the selected cells to Clipboard
- Copy Cells ← Copy cells from Clipboard to current position
- Paste Cells Above → Paste cells above current cell
- Paste Cells Below ← Paste cells below current cell
- Paste Cells & Replace → Paste cells on top of current cell
- Delete Cells ← Delete cells
- Undo Delete Cells → Revert 'Delete cells' invocation
- Split Cell ← Split up cell from current position
- Merge Cell Above → Merge current cell with above
- Merge Cell Below ← Merge current cell with below
- Move Cell Up → Move current cell up
- Move Cell Down ← Move current cell down
- Edit Notebook Metadata → Adjust Metadata underlying the current Notebook
- Find and Replace ← Find and replace in selected cells
- Cut Cell Attachments → Remove cell attachments
- Copy Cell Attachments ← Copy attachments of current cell
- Paste Cell Attachments → Paste attachments of current cell
- Insert Image ← Insert image in selected cells

View Cells

View | Insert | Cell

- Toggle Header → Toggle display of Jupyter logo & Filename
- Toggle Toolbar ← Toggle display of Toolbar
- Toggle Line Numbers → Toggle line numbers in cell
- Cell Toolbar ← Toggle display of cell action icons

Insert Cells

Insert | Cell | Kern

- Insert Cell Above → Add new cell above the current one
- Insert Cell Below ← Add new cell below the current one

Keyboard Shortcuts

Command	Description
enter	enter edit mode
Command + a; Command + c; Command + v	select all; copy; paste
Command + z; Command + y	undo; redo
Command + s	save and checkpoint
Command + b; Command + a	insert cell below; insert cell above
Shift + Enter	run cell, select below
Shift + m	merge cells
Command +]; Command + [	indent; dedent
Ctrl + Enter	run cell
Option + Return	run cell, insert cell below
Escape	enter command mode
Escape + d + d	delete selected cell
Escape + y	change cell to code
Escape + m	change cell to markdown
Escape + r	change cell to raw
Escape + 1	change cell to Heading 1
Escape + n	change cell to heading n
Escape + b	create cell below
Escape + a	Insert cell above

Magic Commands

Statement	Explanation	Example
%magic	Comprehensively lists and explains magic functions	%magic
%automagic	When active, enables you to call magic functions without the '%'	%automagic
%quickref	Launch IPython quick reference	%quickref
%pastebin	Pastebins lines from your current session.	%pastebin 3 18-20 ~1/1-5
%debug	Enters the interactive debugger	%debug
%hist	Print command input and output history	%hist
%pdb	Automatically enter python debugger after any exception	%pdb
%cpaste	Opens up a special prompt for manually pasting Python code for execution	%cpaste
%reset	Delete all variables and names defined in the current namespace	%reset
%run	Run a python script inside a notebook	%run script.py
%who, %who_ls, %whos	Display variables defined in the interactive namespace, with varying levels of verbosity	%who, %who_ls, %whos
%xdel	Delete a variable in the local namespace. Clear any references to that variable	%xdel variable
%time	Times a single statement	In [561]: %time method = [a for a in data if b.startswith('http')]

Execute Cells

Cell | Kernel | Widgets

- Run Cells
- Run Cells and Select Below
- Run Cells and Insert Below
- Run All
- Run All Above
- Run All Below
- Cell Type
- Current Outputs
- All Output

Run Selected Cells; Run Current Cells down & create one below; Run all Cells; Run all Cells above the current one; Toggle & clear current outputs; Run Current Cells down & create one above; Run all Cells below current one; Change the cell type; Toggle & clear all outputs

Kernel Cells

Kernel | Widgets | He

- Interrupt
- Restart
- Restart & Clear Output
- Restart & Run All
- Reconnect
- Shutdown
- Change kernel

Interrupt kernel; Restart Kernel; Interrupt kernel & Clear all output; Restart Kernel & Run all cells; Reconnect to a remote Notebook; Shutdown all cells; Run other installed kernels

Widgets

Widgets | Help

- Save Notebook Widget State
- Clear Notebook Widget State
- Download Widget State
- Embed Widgets

Save Notebook with Interactive widget; Clear Notebook with Interactive widget; Download all widget models in use; Embed current widgets

Help

Help

- User Interface Tour
- Keyboard Shortcuts
- Edit Keyboard Shortcuts
- Notebook Help
- Markdown
- Python Reference
- IPython Reference
- NumPy Reference
- SciPy Reference
- Matplotlib Reference
- SymPy Reference
- pandas Reference
- About

Walk through a UI Tour; Built-in keyboard shortcuts; Edit the Built-in keyboard shortcuts; Notebook help topics; Markdown available in Notebook; Python help topics; IPython help topics; NumPy help topics; SciPy help topics; Matplotlib help topics; SymPy help topics; Pandas help topics; About Jupyter Notebook

Main Area

Activate Next Tab	application:activate-next-tab	Ctrl Shift]
Activate Previous Tab	application:activate-previous-tab	Ctrl Shift [
Toggle Left Area	application:toggle-left-area	Accel B
Toggle Single-Document Mode	application:toggle-mode	Accel Shift D
Toggle Presentation Mode	application:toggle-presentation-mode	(unset)
Toggle Right Area	application:toggle-right-area	(unset)
Activate Command Palette	apputils:activate-command-palette	Accel Shift C
New Launcher	launcher:create	(unset)
Open Keyboard Shortcut Settings	shortcutui:open-ui	Ctrl Shift /
Activate Previously Used Tab	tabmenu:activate-previously-used-tab	Accel Shift `

File Operations

Close Activity	docmanager:close	Ctrl Q
Save	docmanager:save	Accel S
Save As	docmanager:save-as	Accel Shift S
New Launcher	filebrowser:create-main-launcher	Accel Shift L
Toggle Filebrowser	filebrowser:toggle-main	Accel Shift F

Image Viewer

Flip Image Horizontally	imageviewer:flip-horizontal	H
Flip Image Vertically	imageviewer:flip-vertical	V
Invert Colors	imageviewer:invert-colors	I
Reset Image	imageviewer:reset-image	0
Rotate Clockwise	imageviewer:rotate-clockwise	]
Rotate Counterclockwise	imageviewer:rotate-counterclockwise	[
Zoom In	imageviewer:zoom-in	=
Zoom Out	imageviewer:zoom-out	-

Settings Editor

Open Debugger	settingeditor:debug	Accel I
Open Settings Editor	settingeditor:open	Accel ,
Save Settings	settingeditor:save	Accel S

Tooltips

Dismiss Completer	tooltip:dismiss	Escape
Dismiss Completer	tooltip:dismiss	Escape
Invoke Completer	tooltip:launch-console	Shift Tab
Invoke Completer	tooltip:launch-file	Shift Tab
Invoke Completer	tooltip:launch-notebook	Shift Tab

Notebook Operations

Enter Command Mode	notebook:enter-command-mode	Escape
Enter Command Mode	notebook:enter-command-mode	Ctrl M
Enter Edit Mode	notebook:enter-edit-mode	Enter
Show Line Numbers	viewmenu:line-numbering	(unset)
Match Brackets	viewmenu:match-brackets	(unset)
Run Cell(s)	notebook:run-cell	Ctrl Enter
Run Cell and Insert Below	notebook:run-cell-and-insert-below	Alt Enter
Run Cell and Select Next	notebook:run-cell-and-select-next	Shift Enter
Split Cell at Cursor	notebook:split-cell-at-cursor	Ctrl Shift -

Run Menu

Restart Kernel and Run All	runmenu:restart-and-run-all	(unset)
Run	runmenu:run	Shift Enter
Run All	runmenu:run-all	(unset)

Kernel Operations

Interrupt Kernel	kernelmenu:interrupt	I - I
Restart Kernel	kernelmenu:restart	0 - 0
Restart Kernel and Clear	kernelmenu:restart-and-clear	(unset)
Shutdown Kernel	kernelmenu:shutdown	(unset)

Editing

Clear All	editmenu:clear-all	(unset)
Clear	editmenu:clear-current	(unset)
Find	editmenu:find	(unset)
Find and Replace	editmenu:find-and-replace	(unset)
Redo	editmenu:redo	Ctrl Shift Z
Undo	editmenu:undo	Ctrl Z

Console

Insert Line Break	console:linebreak	Enter
Run Cell (forced)	console:run-forced	Shift Enter

Completer

Invoke Console Completion	completer:invoke-console	Tab
Invoke File Completion	completer:invoke-file	Tab
Invoke Notebook Completion	completer:invoke-notebook	Tab

Inspector

Open Inspector	inspector:open	Accel I

Notebook Cell Operations

Change to Code Cell Type	notebook:change-cell-to-code	Y
Change to Markdown Cell Type	notebook:change-cell-to-markdown	M
Change to Raw Cell Type	notebook:change-cell-to-raw	R
Change to Heading 1	notebook:change-cell-to-heading-1	1
Change to Heading 2	notebook:change-cell-to-heading-2	2
Change to Heading 3	notebook:change-cell-to-heading-3	3
Change to Heading 4	notebook:change-cell-to-heading-4	4
Change to Heading 5	notebook:change-cell-to-heading-5	5
Change to Heading 6	notebook:change-cell-to-heading-6	6
Copy Cell(s)	notebook:copy-cell	C
Cut Cell(s)	notebook:cut-cell	X
Delete Cell(s)	notebook:delete-cell	D - D
Extend Selection Above	notebook:extend-marked-cells-above	Shift ArrowUp
Extend Selection Above	notebook:extend-marked-cells-above	Shift K
Extend Selection Below	notebook:extend-marked-cells-below	Shift ArrowDown
Extend Selection Below	notebook:extend-marked-cells-below	Shift J
Insert Cell Above	notebook:insert-cell-above	A
Insert Cell Below	notebook:insert-cell-below	B
Merge Selected Cells	notebook:merge-cells	Shift M
Move Cursor Down	notebook:move-cursor-down	ArrowDown
Move Cursor Down	notebook:move-cursor-down	J
Move Cursor Up	notebook:move-cursor-up	ArrowUp
Move Cursor Up	notebook:move-cursor-up	K
Paste Cell(s) Below	notebook:paste-cell-below	V
Redo Cell Operation	notebook:redo-cell-action	Shift Z
Run Cell(s)	notebook:run-cell	Ctrl Enter
Run Cell(s) and Insert Below	notebook:run-cell-and-insert-below	Alt Enter
Run In Console	notebook:run-in-console	(unset)
Select All Cells	notebook:select-all	Accel A
Toggle All Cell Line Numbers	notebook:toggle-all-cell-line-numbers	Shift L
Toggle Cell Line Numbers	notebook:toggle-cell-line-numbers	L
Undo Cell Operation	notebook:undo-cell-action	Z

Help

Toggle Help	help:toggle	Ctrl Shift H

Source:
Jupiter Cheat sheet | Kaggle

JupyterLab Keyboard Shortcuts

The Essentials

Notebook Operations

- ESC → Enter Command mode
  → Edit the notebook as a whole, but not type into individual cells

  In [1]: a = 10

- Enter → Enter Edit mode
  → Type into the cells like a normal text editor

  In [1]: a = 10

- CTRL + Enter → Run cell
- ALT + Enter → Run cell and insert below
- SHIFT + Enter → Run cell and select next
- CTRL + F → Find

Magic Commands

% = line magic | %% = cell magic | ! = run shell command

- %magic
  List and explain all magic functions
- %time | %timeit
  Place at the top of a cell to check how long the cell takes to execute
- %heat
  Place at the top of a cell to check how long each line of code takes to execute
- %conda | %pip
  Run the conda or pip package manager within the current kernel
- %%python | %%python2 | %%python3
  Run cells with python in a subprocess

Command Mode

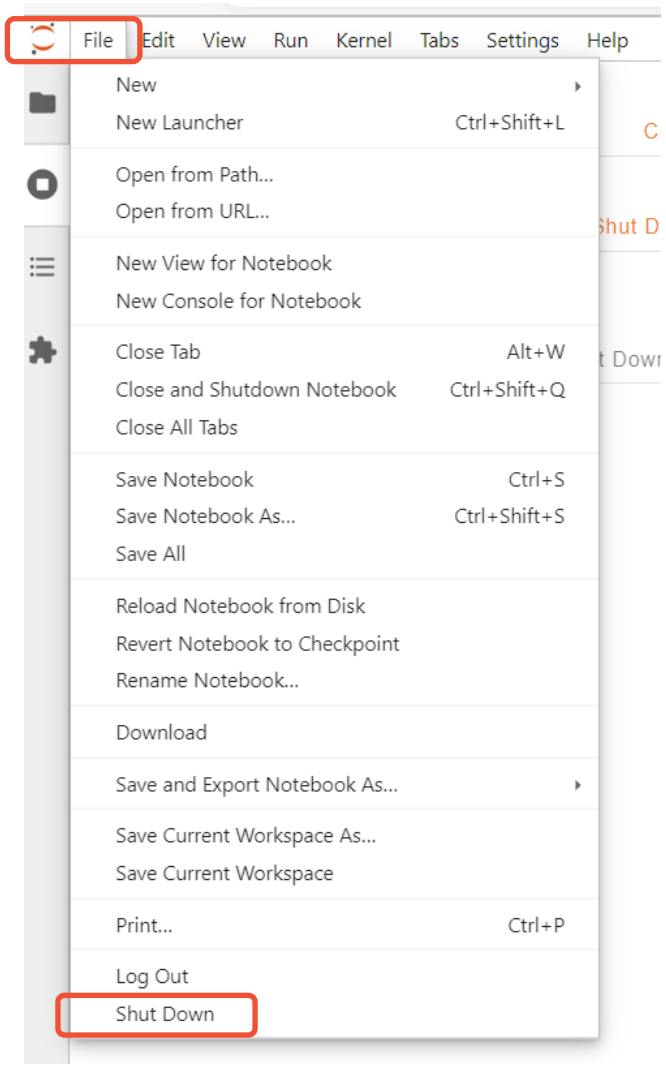
[Esc] to initiate

- ↑ ↓ → Navigate Cells
- A → Insert Cell Above
- B → Insert Cell Below
- D,D → Delete Cell
- O → Toggle Output
- Shift + ↑ ↓ → Select Cells Above/Below
- Shift + M → Merge Multiple Cells
- Ctrl + Shift → Split cell
- y → Change to code
- m → Change to markdown
- r → Change to raw
- c → Copy a cell
- x → Cut a cell
- v → Paste a cell
- z → undo cell operation

JupterLab

Best Practices

- Always exit using **File > Shutdown**



JupyterLab Markdown Language

Basic Syntax

Element	Markdown Syntax

Heading	`# H1`
	`## H2`
	`### H3`

Bold	`**bold text**`
Italic	`*italicized text*`
Blockquote	`> blockquote`

Ordered List	`1. First item`
	`2. Second item`
	`3. Third item`

Unordered List	`- First item`
	`- Second item`
	`- Third item`

Code	`` `code` ``
Horizontal Rule	`---`
Link	`[title](https://www.example.com)`
Image	`![alt text](image.jpg)`

Extended Syntax

Element	Markdown Syntax

Table	`| Syntax | Description |`
	`| ---------- | ---------- |`
	`| Header | Title |`
	`| Paragraph | Text |`

Fenced Code Block	```
	{
	"firstName": "John",
	"lastName": "Smith",
	"age": 25
	}
	``

Footnote	`Here's a sentence with a footnote. [^1]`
	
	`[^1]: This is the footnote.`

Heading ID	`### My Great Heading {#custom-id}`
Definition List	`term` `: definition`
Strikethrough	`~~The world is flat.~~`

Task List	`- [x] Write the press release`
	`- [ ] Update the website`
	`- [ ] Contact the media`

Emoji	`That is so funny! :joy:`
Highlight	`I need to highlight these ==very important words==.`
Subscript	`H~2~O`
Superscript	`X^2^`

Source: Markdown Cheat Sheet | Markdown Guide

Assignment JupLab-01

Hello JupyterLab

- Open JupyterLab and create a new notebook “JupLab-01”
- Change the first cell to type Markdown and add a header “Hello JupyterLab”
- Add a cell below and enter and execute a mathematical equation
- Add a cell above and enter and execute `Print(“Hello there!”)`
  - Did that work? Why? Correct as needed…